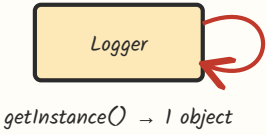
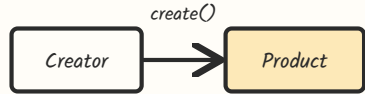
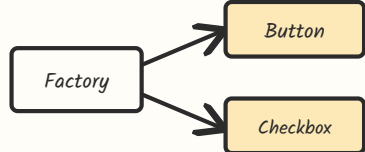
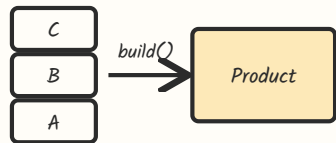
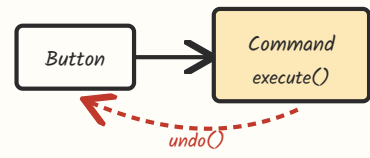
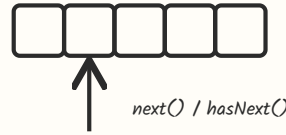
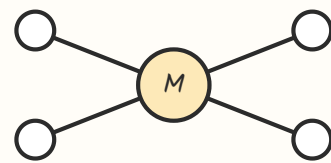
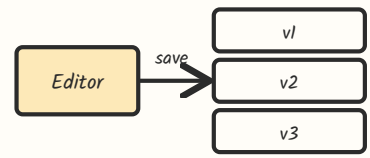
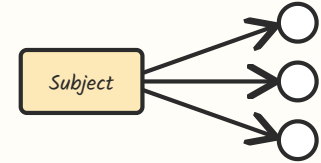
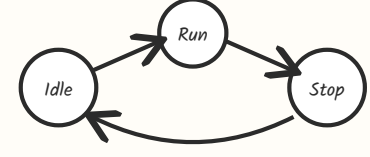
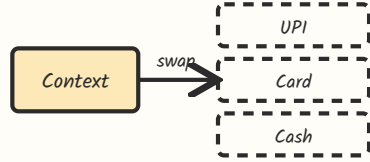
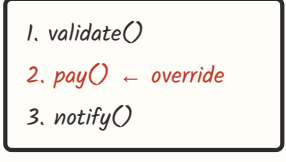


⇒ LLD DESIGN PATTERN RECOGNITION

— RIGHT APPROACH · CLEAN DESIGN —

PATTERN	INTENT	SPOT IT FROM	KEY PARTICIPANTS	USED IN LLD PROBLEMS	IN LARAVEL / NODE	PROS ✓ / CONS ✗	NOT THIS IF → USE
★ CREATIONAL — how objects are created							
1. Singleton 	Ensure a class has only ONE instance and give a global access point.	"Only one" of something - Shared config / logger / pool - Expensive shared resource	Singleton (private constructor, static getInstance())	- Logging Framework - Parking Lot (the lot) - Configuration Manager - Cache Manager - DB Connection Pool	- Laravel: <code>\$app->singleton()</code> - Node: module cache — <code>require()</code> returns same object	✓ One controlled instance ✓ Lazy initialisation ✗ Hidden global state ✗ Hard to unit-test / mock ✗ Thread-safety (double-checked locking)	✗ Need testability / swapping → DI with singleton scope
2. Factory Method 	Create objects without naming the concrete class; subclass / method decides which.	"Based on type, create..." - Big switch / if-else on type - New types added often	- Creator (factoryMethod) - ConcreteCreator - Product, ConcreteProduct	- Notification System (Email/SMS/Push) - Parking Lot vehicle types - Payment methods - Shape / Document editor - Logistics (Truck/Ship)	- Laravel Managers: <code>createRedisDriver()</code> - Node: <code>http.createServer()</code>	✓ Open/Closed for new types ✓ Client decoupled from concrete ✗ Class explosion ✗ Overkill for 1-2 types	✗ Families of related objects → Abstract Factory ✗ Many optional params → Builder
3. Abstract Factory 	Create FAMILIES of related objects that must work together.	"Theme / platform / family" - Products used together - Swap whole family at once	- AbstractFactory, ConcreteFactory - AbstractProduct(s) - ConcreteProduct(s)	- UI Toolkit (Dark / Light) - Cross-platform app (Win / Mac) - DB driver family - Payment gateway family - Furniture shop (Modern / Victorian)	- Laravel DB: Connector + Grammar + Processor per driver - Node: Sequelize dialects	✓ Guarantees compatible products ✓ Swap family in one place ✗ New product type = change every factory ✗ Many interfaces	✗ Only ONE product type → Factory Method
4. Builder 	Construct a complex object step by step; same process, different results.	- Constructor with many optional params - Telescoping constructors - Immutable object built in steps	- Builder, ConcreteBuilder - Director (optional) - Product	- Pizza / Burger order - HTTP Request builder - Query builder - Resume / Report generator - Car configurator	- Laravel Query Builder: <code>->where()->get()</code> - MailMessage-<code>>line()->action()</code> - Node: Knex	✓ Readable fluent API ✓ Validate before build() ✓ Immutability ✗ More code ✗ Builder mirrors product fields	✗ Few params → Plain constructor / named args ✗ Choosing a type → Factory
5. Prototype 	Create new objects by CLONING an existing one.	- Creation is expensive - Many similar objects, small diffs - Runtime-configured templates	- Prototype (clone()) - ConcretePrototype - Registry (optional)	- Game enemy spawning - Document templates - Copy-paste shapes in editor - Chess board state copy - Test data fixtures	- PHP clone + <code>__clone()</code> - Laravel <code>\$model->replicate()</code> - JS <code>structuredClone()</code>	✓ Skips costly init ✓ Add objects at runtime ✗ Shallow vs deep copy bugs ✗ Circular references	✗ Objects are cheap to create → new / Factory
★ STRUCTURAL — how objects are composed							
6. Adapter 	Convert one interface into another that the client expects.	- Third-party / legacy API doesn't fit "Integrate X with Y" - Incompatible interfaces	- Target (interface) - Adapter - Adaptee, Client	- Payment gateways (Stripe / Razorpay) - Legacy system integration - Weather / Maps API - Media player formats 3rd-party logger	- Laravel Filesystem (Flysystem: S3, local) - Node: axios adapters	✓ Reuse code without changing it ✓ Single Responsibility ✗ Extra layer ✗ Many adapters to maintain	✗ Simplify MANY classes → Facade ✗ Add behaviour, same interface → Decorator

PATTERN	INTENT	SPOT IT FROM	KEY PARTICIPANTS	USED IN LLD PROBLEMS	IN LARAVEL / NODE	PROS ✓ / CONS ✗	NOT THIS IF → USE
7. Bridge 	Split abstraction from implementation so BOTH can vary independently.	- Two dimensions → $M \times N$ subclasses - Shape $\times$ Color, Remote $\times$ Device - Designed up-front	- Abstraction, RefinedAbstraction - Implementor - ConcreteImplementor	- Notification type $\times$ channel - Remote control $\times$ devices - Shapes $\times$ renderers - Payment $\times$ gateway - Report $\times$ export format	- Laravel Notifications $\times$ Channels (mail, sms, slack) - Monolog handlers	✓ Avoids $M \times N$ classes ✓ Extend each side independently ✗ More complex up-front ✗ Benefit not obvious early	✗ Fixing a mismatch after the fact → Adapter
8. Composite 	Treat single objects and groups the SAME way in a tree.	- Part-whole hierarchy - Folders & files, menus & sub-menus - Recursive totals (size / price)	- Component - Leaf - Composite (children list)	- File System - Org hierarchy / Employees - Menu / Category tree - UI component tree - Order with bundles	- Laravel Blade nested components - Node: React component tree, DOM	✓ Uniform treatment ✓ Recursion is natural ✗ Hard to restrict child types ✗ add() meaningless on leaves	✗ No hierarchy, just wrapping → Decorator
9. Decorator 	Add responsibilities at RUNTIME by wrapping; interface stays the same.	- Add-ons in combinations - Avoid subclass explosion "Wrap" a feature around	- Component, ConcreteComponent - Decorator (wraps Component) - ConcreteDecorator	- Coffee / Pizza toppings pricing - Discount stacking - Text formatting - Stream compress + encrypt - Caching / logging wrapper	- Laravel \$app->extend() to wrap a service - Node: higher-order functions, TS @decorators	✓ Mix features at runtime ✓ Open/Closed + SRP ✗ Many tiny objects ✗ Wrap order matters ✗ Layered debugging	✗ Change the interface → Adapter ✗ Control access → Proxy
10. Facade 	One simple entry point to a complex subsystem.	"Simplify" - Client calls many classes for one task - Single entry: placeOrder()	- Facade - Subsystem classes - Client	- Order placement (stock + pay + ship) - Movie / Hotel booking service - Home theater - Computer boot - Video conversion	- Laravel Facades: Cache::get(), DB::table() - Node: SDK wrapper classes	✓ Decouples client from subsystem ✓ Easier API ✗ Can grow into a God object ✗ Hides advanced features	✗ Convert one interface → Adapter ✗ Peers talk to each other → Mediator
11. Flyweight 	SHARE common state among huge numbers of objects to save memory.	- Millions of similar objects - Memory is the bottleneck - State = shared + unique	- Flyweight (intrinsic state) - FlyweightFactory (cache) - Client (extrinsic state)	- Text editor characters - Game trees / bullets - Chess pieces - Map markers - Icon cache	- Rare in web apps - PHP / JS string interning, V8 hidden classes	✓ Huge memory savings ✗ More complexity ✗ Extrinsic state passed around ✗ Shared state thread-safety	✗ Few objects → Skip it ✗ Reuse whole objects → Object Pool
12. Proxy 	Stand-in that CONTROLS access to a real object (lazy, cache, auth, remote).	- Lazy-load expensive object - Access control / permissions - Caching, logging, remote calls	- Subject (interface) - RealSubject - Proxy	- Image lazy loading - Document access control - API caching layer - Rate-limiter wrapper - ATM → Bank	- Eloquent lazy-loaded relations - Node: JS Proxy, Vue reactivity	✓ Control without touching real object ✓ Lazy / secure / cached ✗ Extra latency ✗ Stale cache risk	✗ Adding features → Decorator ✗ Simplifying many → Facade
★ BEHAVIORAL — how objects communicate							
13. Chain of Responsibility 	Pass a request along a chain until a handler deals with it.	- Ordered handlers / levels - Approvals, escalation - Filters / middleware	- Handler (setNext, handle) - ConcreteHandler - Client	- ATM cash dispenser (2000/500/100) - Logger levels - Expense approval - Support ticket escalation - Request validation pipeline	- Laravel Middleware & Pipeline - Express / Koa middleware next()	✓ Sender decoupled from receiver ✓ Reorder / add handlers easily ✗ Request may go unhandled ✗ Long chains hard to debug	✗ ALL listeners must hear it → Observer

PATTERN	INTENT	SPOT IT FROM	KEY PARTICIPANTS	USED IN LLD PROBLEMS	IN LARAVEL / NODE	PROS ✓ / CONS ✗	NOT THIS IF → USE
14. Command 	Wrap a request as an OBJECT — queue it, log it, undo it.	- Undo / redo - Queue or schedule actions - Macros, remote buttons	- Command (execute / undo) - ConcreteCommand, Receiver - Invoker, Client	- Text editor undo / redo - Remote control - Job scheduler / queue - Restaurant order system - Stock trade orders	- Laravel Jobs, Artisan commands, Command Bus - Node: Redux actions, BullMQ jobs	✓ Undo / redo ✓ Queue & log requests ✗ Many small classes ✗ Undo state costs memory	✗ Swap an algorithm → Strategy ✗ Snapshot full state → Memento
15. Iterator 	Walk a collection one by one without exposing its internals.	- Traverse a custom collection - Several traversal orders - Hide representation	- Iterator (next / hasNext) - ConcreteIterator - Aggregate / Collection	- Playlist / Music player - Feed pagination - Tree traversal (BFS / DFS) - Menu items - Library catalogue	- PHP IteratorAggregate, yield - Laravel LazyCollection, cursor() - JS Symbol.iterator	✓ Uniform traversal ✓ Lazy, many iterators at once ✗ Overkill for plain arrays ✗ Concurrent modification	✗ Different ops per element type → Visitor
16. Mediator 	Centralise communication so objects don't talk to each other directly.	- Many-to-many interactions - Chat room, control tower - Tightly coupled components	- Mediator, ConcreteMediator - Colleagues	- Chat Room - Air Traffic Control - Auction System - Elevator dispatcher - UI form dialog	- Laravel event bus (partly) - Node: Socket.io rooms, message broker	✓ Coupling $N^2 \rightarrow N$ ✓ Logic in one place ✗ Mediator becomes a God object	✗ One-to-many broadcast → Observer
17. Memento 	Capture & restore an object's state without breaking encapsulation.	- Undo / rollback - Snapshots, checkpoints - Save game, history	- Originator - Memento (snapshot) - Caretaker (history)	- Text editor undo - Game save / checkpoint - Drawing canvas history - Form draft restore - Config rollback	- Eloquent getOriginal(), revision packages - Node: Redux time-travel	✓ Keeps encapsulation ✓ Simple undo ✗ Memory heavy for big state ✗ Caretaker lifecycle	✗ Undo by reversing actions → Command
18. Observer 	One-to-many: when the subject changes, all subscribers are notified.	"Notify / subscribe / on event" - Real-time updates - Pub-sub	- Subject (attach / detach / notify) - Observer (update) - ConcreteObserver	- Notification Service - Stock price ticker - YouTube subscribe - Auction bids - Order status updates	- Laravel Events / Listeners, Eloquent Observers - Node: EventEmitter, RxJS	✓ Loose coupling ✓ Add subscribers freely ✗ Unpredictable order ✗ Leaks if not detached ✗ Cascading updates	✗ Exactly ONE handler should act → Chain of Resp. ✗ Complex many-to-many → Mediator
19. State 	Behaviour changes with internal state — as if the class changed.	- Big switch on status - State machine / transitions - Behaviour depends on mode	- Context - State (interface) - ConcreteState (+ transitions)	- Vending Machine - Elevator - ATM - Traffic Light - Order lifecycle	- spatie/laravel-model-states, PHP enums - Node: XState	✓ No giant conditionals ✓ Each state isolated ✗ Many classes ✗ Transitions scattered	✗ Client picks algo, no transitions → Strategy
20. Strategy 	Family of interchangeable algorithms, chosen at RUNTIME.	- Many ways to do one task - if-else on algorithm type - Pricing / payment / routing options	- Context - Strategy (interface) - ConcreteStrategy	- Payment methods - Parking fee calculation - Discount / pricing rules - Route finding (car / bike / walk) - Rate-limiter algorithm	- Laravel drivers (cache, queue, session), Auth guards - Node: Passport strategies	✓ Open/Closed ✓ Swap at runtime, easy to test ✗ Client must know strategies ✗ More classes	✗ Changes via internal transitions → State ✗ Fixed steps, some vary → Template Method
21. Template Method 	Base class fixes the algorithm skeleton; subclasses fill steps.	- Same steps, different details - Duplicate code in siblings - Hooks	- AbstractClass (templateMethod, hooks) - ConcreteClass	- Data parser (CSV / JSON / XML) - Payment flow (validate → pay → notify) - Report generation - Game loop - Beverage maker	- Laravel Migration up() / down(), TestCase setUp() - Node: class lifecycle hooks	✓ Code reuse ✓ Skeleton stays in control ✗ Inheritance coupling ✗ Rigid, LSP risk	✗ Swap the whole algo (composition) → Strategy

PATTERN	INTENT	SPOT IT FROM	KEY PARTICIPANTS	USED IN LLD PROBLEMS	IN LARAVEL / NODE	PROS ✓ / CONS ✗	NOT THIS IF → USE
22. Visitor 	Add NEW operations to a class hierarchy without changing the classes.	- Many operations over stable types - Export / tax / render per type - Double dispatch	- Visitor (visitX per element) - ConcreteVisitor - Element (accept)	- Cart tax per item type - Document export (PDF / HTML) - Compiler AST passes - File system scan - Insurance per building type	- PHP-Parser NodeVisitor - Node: Babel / ESLint visitors	✓ Easy to add operations ✓ Related ops in one class ✗ New element type breaks all visitors ✗ Weakens encapsulation	✗ Element types change often → Polymorphism
23. Interpreter 	Define a grammar and evaluate sentences in it.	- Mini language / DSL - Rule engine - Parse & evaluate expressions	- AbstractExpression - Terminal / NonTerminal Expression - Context	- Calculator - Discount rule engine - Search query parser - SQL-like filters - Boolean feature flags	- Blade compiler, rules 'required max:255' - Node: template engines, math.js	✓ Easy to extend grammar ✗ Class per rule explodes ✗ Slow for big grammars	✗ Grammar is complex → Parser generator
★ BEYOND GoF — used in almost every LLD round							
24. Dependency Injection 	Pass dependencies IN instead of creating them inside.	new inside business classes - Hard to unit-test - Swappable implementations	- Client - Service interface - Injector / Container	- EVERY LLD answer - Services depend on interfaces - Repo, Notifier, PaymentGateway - Mock in tests - Swap providers	- Laravel Service Container, constructor injection - Node: NestJS DI, InversifyJS	✓ Testable, loose coupling ✓ Dependency Inversion (D) ✗ Container "magic" ✗ Wiring errors at runtime	✗ Tiny script, no variation → Direct new
25. Repository 	Collection-like interface between domain and data storage.	SQL mixed with business logic - Swap storage (DB / cache / memory) - Test without a DB	- Repository interface - ConcreteRepository - Entity, Data source	- Library Management - Parking Lot tickets - Splitwise - BookMyShow - Hotel Booking	- Laravel repositories over Eloquent - Node: TypeORM / Prisma repositories	✓ Testable, persistence isolated ✓ DDD friendly ✗ Extra layer over ORM ✗ Leaky (N+1, pagination)	✗ Simple CRUD → Eloquent model directly
26. Null Object 	Do-nothing default object instead of null.	- Repeated if (x !== null) - Optional collaborator - Default behaviour	- AbstractObject - RealObject - NullObject	- No-discount coupon - Guest user - Null logger - Default notifier - Empty cart item	- Laravel optional(), relation withDefault() - Monolog NullHandler, Node noop fn	✓ Removes null checks ✓ Safer calls ✗ Can silently hide real errors	✗ Absence is an error → Throw exception

SOLID — THE WHY BEHIND EVERY PATTERN				
S Single Responsibility One reason to change e.g. Split Invoice / InvoicePrinter	O Open / Closed Extend, don't modify e.g. Strategy, Decorator	L Liskov Substitution Child must work wherever parent does e.g. No throwing overrides (Penguin.fly)	I Interface Segregation Small focused interfaces e.g. Printable, Scannable — not IMachine	D Dependency Inversion Depend on abstractions e.g. Dependency Injection

UML RELATIONSHIPS — FOR YOUR CLASS DIAGRAM					
Inheritance is-a (extends) Car → Vehicle	Implementation implements interface UpiPay → Payment	Association uses / knows (field) Driver → Car	Aggregation has-a, lives alone Team ◇— Player	Composition owns, dies with parent House ◆— Room	Dependency uses temporarily (param) Order → Printer

TOP LLD INTERVIEW PROBLEMS → PATTERNS TO MENTION					
Parking Lot <i>Singleton · Factory · Strategy (fee)</i>	Elevator <i>State · Strategy (scheduling) · Observer</i>	Vending Machine / ATM <i>State · Chain (cash dispense)</i>	BookMyShow <i>Facade · Observer · seat locking</i>	Splitwise <i>Strategy (split types) · Observer</i>	Chess / Tic-Tac-Toe <i>Factory · Command (undo) · Strategy</i>
Snake & Ladder <i>Factory · Strategy (dice) · Singleton</i>	Rate Limiter <i>Strategy (token / leaky bucket)</i>	Logger <i>Singleton · Chain of Resp. · Observer</i>	Cab Booking (Uber) <i>Strategy (match / price) · State · Observer</i>	Notification Service <i>Observer · Factory · Bridge</i>	LRU Cache <i>HashMap + DLL · Strategy (eviction)</i>

LLD INTERVIEW FLOW — SAY IT OUT LOUD					
1 <i>Clarify requirements & scope (functional + non-functional)</i>	2 <i>Find entities (nouns) & actions (verbs)</i>	3 <i>Draw class diagram: is-a / has-a</i>	4 <i>Apply a pattern ONLY where it solves a real problem</i>	5 <i>Code interfaces & core classes first</i>	6 <i>Discuss concurrency, extensibility, edge cases</i>

